Sentiment Analysis Shopee Reviews

1. Import Library dan Load Dataset

Dataset yang digunakan merupakan hasil scraping mandiri dari Google Play Store yang telah melalui proses cleaning, labeling, dan balancing.

Dataset terdiri dari:

- 12.000 data ulasan
- 3 kelas sentimen (negatif, netral, positif)
- Distribusi seimbang (masing-masing 4000 data)

Dataset ini siap digunakan untuk proses modeling tanpa memerlukan penanganan missing value tambahan.

```
!pip install -U transformers datasets accelerate -q
import pandas as pd
import numpy as np

# load dataset final
df = pd.read_csv('shopee_reviews_final.csv')

# preview
df.head()
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/10.1 MB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.1/10.1 MB 382.8 MB/s eta
0:00:01 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.1/10.1 MB 186.8
MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 527.0/527.0 kB 60.8 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47.6/47.6 MB 64.9 MB/s eta
0:00:00
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 12000,\n  \"fields\":
[\n    {\n      \"column\": \"review\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 11318,\n
\"samples\": [\n          \"Poko nya mantab jaya\",\n
\"sanagat mudah d gunakan , toko toko kompetitif,,,, jafi banayk
pilihan\",\n          \"min gmna nih tanggapan Mimin tentang driver
shoppe yg nabrak plat di Bondongan Bogor mobil F 5847 AAO\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"rating\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1,\n        \"min\": 1,\n        \"max\": 5,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          3,\n
```

2,\n   5\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"sentiment\",\n   \"properties\": {\n   \"dtype\":
\"category\",\n   \"num_unique_values\": 3,\n   \"samples\":
[\n   \"negatif\",\n   \"netral\",\n
\"positif\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"df"}

2. Exploratory Data Analysis (EDA)

Dilakukan pengecekan struktur data, missing value, dan distribusi kelas.

Hasil:

- Tidak terdapat missing value
- Dataset memiliki distribusi yang seimbang (4000 per kelas)

Insight: Distribusi yang seimbang sangat penting untuk menghindari bias model terhadap kelas tertentu, terutama pada kasus multi-class classification.

```
# info dataset
df.info()

# cek missing value
print("\nmissing values:")
print(df.isnull().sum())

# distribusi kelas
print("\ndistribusi sentimen:")
print(df['sentiment'].value_counts())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12000 entries, 0 to 11999
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   review     12000 non-null  object
 1   rating     12000 non-null  int64
 2   sentiment  12000 non-null  object
dtypes: int64(1), object(2)
memory usage: 281.4+ KB

missing values:
review       0
rating       0
sentiment    0
dtype: int64

distribusi sentimen:
```

```
sentiment
negatif    4000
netral     4000
positif    4000
Name: count, dtype: int64
```

3. Text Preprocessing

Preprocessing dilakukan untuk meningkatkan kualitas teks tanpa menghilangkan konteks penting.

Tahapan:

- Lowercasing
- Menghapus URL dan angka
- Mengganti karakter non-alfabet dengan spasi
- Normalisasi spasi

Insight: Pendekatan ini bersifat minimalis agar tetap mempertahankan informasi semantik, terutama penting untuk model berbasis deep learning dan transformer.

```
import re

# fungsi cleaning teks (fixed version)
def clean_text(text):
    text = text.lower()  # lowercase

    text = re.sub(r'http\S+|www\S+|https\S+', '', text)  # hapus url
    text = re.sub(r'\d+', '', text)  # hapus angka

    # □ ganti simbol dengan spasi (bukan hapus langsung)
    text = re.sub(r'[^a-zA-Z!?]', ' ', text)

    # rapikan spasi
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# apply ulang
df['clean_text'] = df['review'].apply(clean_text)

# cek hasil
df[['review', 'clean_text']].head()
```

{"summary":"{\n \"name\": \"df[['review', 'clean_text']]\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"review\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"saya penolakan pernah menolak pesanan di shopee tapi kenapa kurir mengirim bukti tidak barang,buat pihak Shopee tolong di perhatikan

kurirnya,kami pembeli merasa di rugikan karena barang yg kami pesan tidak sampai ke tangan kami\",\n   \"buruk sekali\",\n \"Sangat ribet, selalu tidak bisa login padahal biasanya aman-aman saja. Parah ! Nggak lagi-lagi deh pakai aplikasi yang jelek seperti ini, sangat berbahaya di saat dibutuhkan untuk transfer malah aplikasinya TIDAK BISA LOGIN dengan keterangan \\\"ERROR -silakan perbarui aplikasi\\\" Dan sudah capek-capek aplikasi diperbarui kendala tetap tak teratasi, SELALU ERIOR TAK BISA LOGIN. Sudah ah mau saya tutup saja akun seabank saya daripada membahayakan diri sendiri.\"\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"clean_text\",\n   \"properties\": {\n   \"dtype\": \"string\",\n   \"num_unique_values\": 5,\n   \"samples\": [\n   \"saya penolakan pernah menolak pesanan di shopee tapi kenapa kurir mengirim bukti tidak barang buat pihak shopee tolong di perhatikan kurirnya kami pembeli merasa di rugikan karena barang yg kami pesan tidak sampai ke tangan kami\",\n   \"buruk sekali\",\n   \"sangat ribet selalu tidak bisa login padahal biasanya aman aman saja parah ! nggak lagi lagi deh pakai aplikasi yang jelek seperti ini sangat berbahaya di saat dibutuhkan untuk transfer malah aplikasinya tidak bisa login dengan keterangan error silakan perbarui aplikasi dan sudah capek capek aplikasi diperbarui kendala tetap tak teratasi selalu erior tak bisa login sudah ah mau saya tutup saja akun seabank saya daripada membahayakan diri sendiri\"\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

4. Persiapan Data untuk Modeling

Dilakukan encoding label dan pembagian data menggunakan stratified split.

Hasil:

- Train: 9600 data
- Test: 2400 data
- Distribusi tetap seimbang di kedua set

Insight: Stratified split memastikan distribusi kelas tetap konsisten, sehingga evaluasi model lebih representatif.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# encode label
le = LabelEncoder()
df['label'] = le.fit_transform(df['sentiment'])

# fitur dan target
X = df['clean_text']
```

```
y = df['label']

# split data (stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# cek ukuran
print("train:", X_train.shape)
print("test:", X_test.shape)

# cek distribusi label train
print("\ndistribusi train:")
print(y_train.value_counts())
```

```
train: (9600,)
test: (2400,)

distribusi train:
label
2    3200
0    3200
1    3200
Name: count, dtype: int64
```

5. Ekstraksi Fitur dengan TF-IDF

Digunakan kombinasi:

- Word-level TF-IDF (n-gram 1–2)
- Character-level TF-IDF (n-gram 3–5)

Total fitur: 20.000

Insight:

- Word-level menangkap konteks kata
- Char-level membantu menangani typo dan slang

Pendekatan ini meningkatkan robustness terhadap data noisy seperti ulasan pengguna.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from scipy.sparse import hstack

# word-level tfidf
tfidf_word = TfidfVectorizer(
    max_features=15000,
```

```
    ngram_range=(1,2),
    min_df=2,
    max_df=0.9
)

# char-level tfidf (penting untuk typo & slang)
tfidf_char = TfidfVectorizer(
    analyzer='char',
    ngram_range=(3,5),
    max_features=5000
)

# fit
X_train_word = tfidf_word.fit_transform(X_train)
X_test_word = tfidf_word.transform(X_test)

X_train_char = tfidf_char.fit_transform(X_train)
X_test_char = tfidf_char.transform(X_test)

# gabungkan fitur
X_train_tfidf = hstack([X_train_word, X_train_char])
X_test_tfidf = hstack([X_test_word, X_test_char])

print("train shape:", X_train_tfidf.shape)
print("test shape:", X_test_tfidf.shape)
```

```
train shape: (9600, 20000)
test shape: (2400, 20000)
```

6. Modeling dengan Logistic Regression

Model Logistic Regression digunakan sebagai baseline karena efektif pada data berbasis TF-IDF.

```
from sklearn.linear_model import LogisticRegression

model_lr = LogisticRegression(
    max_iter=2000,
    C=3.0,
    solver='lbfgs',
    multi_class='auto'
)

model_lr.fit(X_train_tfidf, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/
_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in
version 1.5 and will be removed in 1.7. From then on, it will always
use 'multinomial'. Leave it to its default value to avoid this
warning.
  warnings.warn(
```

```
LogisticRegression(C=3.0, max_iter=2000, multi_class='auto')
```

7. Evaluasi Model

Hasil:

- Accuracy: 0.88
- Macro F1: 0.88

Insight: Model baseline menunjukkan performa yang sangat kuat dan stabil pada semua kelas.

Menariknya, pendekatan tradisional (TF-IDF + LR) mampu memberikan performa tinggi karena:

- Representasi fitur sangat informatif
- Dataset relatif bersih dan seimbang

```
from sklearn.metrics import accuracy_score, classification_report

# prediksi
y_pred = model_lr.predict(X_test_tfidf)

# evaluasi
print("accuracy:", accuracy_score(y_test, y_pred))
print("\nclassification report:\n")
print(classification_report(y_test, y_pred))
```

```
accuracy: 0.8820833333333333

classification report:

              precision    recall  f1-score   support

           0       0.87      0.86      0.86       800
           1       0.86      0.88      0.87       800
           2       0.92      0.91      0.91       800

    accuracy                           0.88      2400
   macro avg       0.88      0.88      0.88      2400
weighted avg       0.88      0.88      0.88      2400
```

Model tradisional mencapai performa yang cukup baik dengan macro F1 sebesar 0.88, menunjukkan kemampuan klasifikasi yang seimbang pada semua kelas

8. Tokenisasi dan Padding

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# parameter
vocab_size = 20000
```

```
max_len = 100

# tokenizer
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

# convert ke sequence
X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

# padding
X_train_pad = pad_sequences(X_train_seq, maxlen=max_len,
padding='post', truncating='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=max_len, padding='post',
truncating='post')

# cek shape
print("train shape:", X_train_pad.shape)
print("test shape:", X_test_pad.shape)
```

```
train shape: (9600, 100)
test shape: (2400, 100)
```

9. Encoding Label untuk Deep Learning

```
from tensorflow.keras.utils import to_categorical

# one-hot encoding
y_train_cat = to_categorical(y_train)
y_test_cat = to_categorical(y_test)

print("y_train shape:", y_train_cat.shape)
```

```
y_train shape: (9600, 3)
```

10. Class Weight

Karena dataset sudah seimbang, nilai class weight = 1 untuk semua kelas.

Insight: Tidak ada penalti khusus untuk kelas tertentu, sehingga model belajar secara proporsional.

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

classes = np.unique(y_train)

class_weights = compute_class_weight(
    class_weight='balanced',
    classes=classes,
```

```
    y=y_train
)

class_weights = dict(enumerate(class_weights))
print(class_weights)
```

```
{0: np.float64(1.0), 1: np.float64(1.0), 2: np.float64(1.0)}
```

11. Model Deep Learning (BiLSTM)

Model BiLSTM digunakan untuk menangkap konteks dua arah dalam teks.

Arsitektur:

- Embedding
- Bidirectional LSTM
- Dropout
- Dense layer

Model ini diharapkan mampu memahami konteks lebih baik dibanding model tradisional.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout,
Bidirectional
from tensorflow.keras.callbacks import EarlyStopping

model_lstm = Sequential([
    Embedding(input_dim=vocab_size, output_dim=128),

    Bidirectional(LSTM(64)),

    Dropout(0.3),

    Dense(64, activation='relu'),
    Dense(3, activation='softmax')
])

model_lstm.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

model_lstm.summary()
```

```
Model: "sequential"

┏━━━
```

```
┃ Layer (type)                    ┃ Output Shape           ┃
Param # ┃
┡━━━
```

```
    validation_data=(X_test_pad, y_test_cat),
    class_weight=class_weights,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/10
300/300 ━━━━━━━━━━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.6453 - loss:
0.7606 - val_accuracy: 0.7421 - val_loss: 0.6199
Epoch 2/10
300/300 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.8546 - loss:
0.4056 - val_accuracy: 0.8383 - val_loss: 0.4708
Epoch 3/10
300/300 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9165 - loss:
0.2411 - val_accuracy: 0.8608 - val_loss: 0.4162
Epoch 4/10
300/300 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9517 - loss:
0.1529 - val_accuracy: 0.8742 - val_loss: 0.4303
Epoch 5/10
300/300 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9624 - loss:
0.1162 - val_accuracy: 0.8858 - val_loss: 0.4280
```

13. Evaluasi Model LSTM

Hasil:

- Accuracy: 0.86
- Macro F1: 0.86

Insight: Model deep learning tidak outperform baseline.

Analisis:

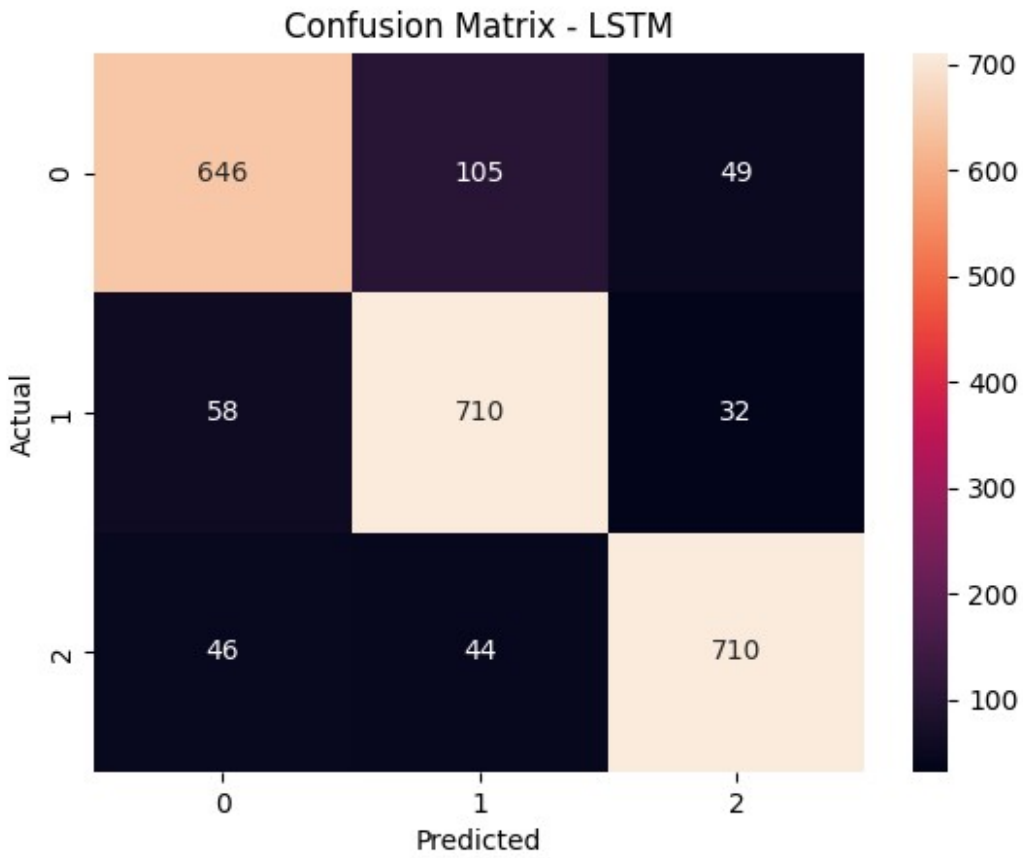
- LSTM membutuhkan data lebih besar untuk optimal
- TF-IDF sudah sangat representatif untuk task ini
- Kelas netral tetap menjadi bottleneck

Kesimpulan: Untuk dataset ini, model tradisional masih lebih efektif dibanding LSTM.

```
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# prediksi
y_pred_dl = model_lstm.predict(X_test_pad)
y_pred_dl = np.argmax(y_pred_dl, axis=1)
```

```python
# accuracy + classification report
print("accuracy:", accuracy_score(y_test, y_pred_dl))
print("\nclassification report:\n")
print(classification_report(y_test, y_pred_dl))

# confusion matrix
cm = confusion_matrix(y_test, y_pred_dl)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - LSTM")
plt.show()

# error analysis (sample)
df_eval = pd.DataFrame({
    'text': X_test,
    'true': y_test,
    'pred': y_pred_dl
})

df_wrong = df_eval[df_eval['true'] != df_eval['pred']]
print("\ncontoh kesalahan prediksi:")
display(df_wrong.head(10))
```

```
75/75 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
accuracy: 0.8608333333333333

classification report:

              precision    recall  f1-score   support

           0       0.86      0.81      0.83       800
           1       0.83      0.89      0.86       800
           2       0.90      0.89      0.89       800

    accuracy                           0.86      2400
   macro avg       0.86      0.86      0.86      2400
weighted avg       0.86      0.86      0.86      2400
```



contoh kesalahan prediksi:

{"summary":"{\n \"name\": \"display(df_wrong\",\n \"rows\": 10,\n \"fields\": [\n {\n \"column\": \"text\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 10,\n \"samples\": [\n \"pengirimannya kadang cepat kadang lambat tapi puas untuk pelayanannya\",\n \"hampir tiap hari belanja di shopee banyak diskonnya n ragam barang dr yg branded smp yg biasa tumpah ruah di shopee cm kenapa ya hari ini aku kok ga bs co ad peringatan dibatasi krna shopeelater sudah limit pdhl aku ga prnh buka later samsek mau transf sesama shopee jg ga bs apa lg arror kah?\",\n \"aplikasinya berat banget\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"true\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 0,\n \"max\": 2,\n \"num_unique_values\": 3,\n \"samples\": [\n 1,\n 2,\n 0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"pred\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 0,\n \"max\": 2,\n \"num_unique_values\": 3,\n \"samples\":

[\n   0,\n   1,\n   2\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

Hasil evaluasi menunjukkan bahwa model LSTM mampu melakukan klasifikasi dengan cukup baik pada seluruh kelas dengan accuracy sebesar 0.86 dan macro F1-score sebesar 0.86.

Model menunjukkan performa terbaik pada kelas positif dengan F1-score sebesar 0.89, yang mengindikasikan bahwa pola sentimen positif relatif lebih mudah dikenali.

Pada kelas negatif, model juga menunjukkan performa yang cukup baik dengan F1-score sebesar 0.83, meskipun masih terdapat kesalahan klasifikasi ke kelas netral.

Kelas netral menjadi tantangan utama dengan F1-score sebesar 0.86. Hal ini terlihat dari confusion matrix di mana banyak data netral yang diprediksi sebagai negatif maupun positif. Fenomena ini menunjukkan bahwa kelas netral memiliki batas yang tidak jelas dan cenderung ambigu secara semantik.

Selain itu, analisis kesalahan prediksi menunjukkan bahwa banyak teks yang mengandung campuran opini (misalnya keluhan disertai pujian) sehingga sulit diklasifikasikan secara tegas ke dalam satu kategori.

Secara keseluruhan, model deep learning mampu menangkap pola sentimen dengan baik, namun masih memiliki keterbatasan dalam menangani ambiguitas pada kelas netral.

14. Fine-Tuning IndoBERT

Digunakan model pre-trained IndoBERT untuk meningkatkan performa dengan pemahaman konteks bahasa Indonesia.

Tambahan:

- Hard negative mining (150 sampel)
- Tokenization dengan max_length=128

Insight: Hard negative mining membantu model belajar dari kesalahan sebelumnya sehingga meningkatkan generalisasi.

```
# import
import pandas as pd
import numpy as np
import torch

from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
```

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from datasets import Dataset

# buat df_bert dari dataframe utama (yang sekarang sudah termasuk data augmentasi)
df_bert = df[['review', 'label']].copy()
df_bert.columns = ['text', 'label']

# hard negative mining: tambahkan sampel yang salah klasifikasi dari evaluasi model sebelumnya
# df_wrong berasal dari evaluasi model lstm (cell jb9gzxycp-ug)
# berisi sampel yang salah klasifikasi (teks, label asli, label prediksi)
if 'df_wrong' in globals():
    # □ ambil sebagian kecil saja dari data yang salah prediksi (agar tidak terlalu banyak noise)
    df_hard_mining_sampled = df_wrong.sample(n=min(150, len(df_wrong)), random_state=42)
    df_hard_mining = df_hard_mining_sampled[['text', 'true']].copy()
    df_hard_mining.rename(columns={'true': 'label'}, inplace=True)
    print(f"menambahkan {len(df_hard_mining)} sampel hard negative ke data training.")
    df_bert = pd.concat([df_bert, df_hard_mining], ignore_index=True)
    print(f"ukuran df_bert baru setelah hard negative mining: {len(df_bert)}")

# split data untuk fine-tuning bert
train_df, test_df = train_test_split(
    df_bert,
    test_size=0.2,
    stratify=df_bert['label'],
    random_state=42
)

# konversi ke format dataset huggingface
train_dataset = Dataset.from_pandas(train_df.reset_index(drop=True))
test_dataset = Dataset.from_pandas(test_df.reset_index(drop=True))
```

```
menambahkan 150 sampel hard negative ke data training.
ukuran df_bert baru setelah hard negative mining: 12150
```

```python
from transformers import AutoTokenizer

# □ gunakan model IndoBERT (lebih cocok untuk bahasa dan domain)
model_name = "indobenchmark/indobert-base-p1"

# load tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```python
# fungsi tokenisasi
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=128   # □ cukup 128 untuk dataset ini (tidak
overkill)
    )

# apply ke dataset
train_dataset = train_dataset.map(
    tokenize_function,
    batched=True,
    remove_columns=["text"]  # □ bersihkan kolom lama
)

test_dataset = test_dataset.map(
    tokenize_function,
    batched=True,
    remove_columns=["text"]
)

# set format ke PyTorch
train_dataset.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "label"]
)

test_dataset.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "label"]
)

print("□ Tokenization selesai tanpa error")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```

{"model_id":"7a009726f5de4079a944fd35c6f53596","version_major":2,"version_minor":0}

{"model_id":"cdac7819b365443c9c4461a7b5027d39","version_major":2,"version_minor":0}

{"model_id":"848b5945043e454bbd269455f8cf9013","version_major":2,"version_minor":0}

{"model_id":"769303bd92ea43a79d8f412d4e138339","version_major":2,"version_minor":0}

{"model_id":"d317019eabba4faaa3d4d54156debedc","version_major":2,"version_minor":0}

{"model_id":"08f85309cfa140108e0738333bc8bc7e","version_major":2,"version_minor":0}

```
□ Tokenization selesai tanpa error
```

```python
import torch
from transformers import AutoConfig,
AutoModelForSequenceClassification

# load config + tambahkan parameter penting
config = AutoConfig.from_pretrained(
    model_name,
    num_labels=3,
    # □ hapus dropout eksplisit, biarkan default model pre-trained
    problem_type="single_label_classification"
)

# load model
model_bert = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    config=config
)

# kirim ke GPU (H100)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_bert.to(device)

print("□ Model loaded & moved to GPU")
```

```
You passed `num_labels=3` which is incompatible to the `id2label` map
of length `5`.
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

{"model_id":"7824c4a8683248f7923f08fe20568521","version_major":2,"version_minor":0}

{"model_id":"514b63c64d2748ab8e99cec75487d388","version_major":2,"version_minor":0}

```
BertForSequenceClassification LOAD REPORT from:
indobenchmark/indobert-base-p1
Key                | Status  |
------------------+---------+-
classifier.bias   | MISSING |
classifier.weight | MISSING |

Notes:
- MISSING: those params were newly initialized because missing from
the checkpoint. Consider training on your downstream task.

 Model loaded & moved to GPU
```

```
# ==========================================
# TRAINING ARGUMENTS (OPTIMIZED)
# ==========================================

from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./results",

    # evaluation
    eval_strategy="epoch",
    save_strategy="epoch",

    # optimizer
    learning_rate=2e-5,
    #  hapus lr_scheduler_type dan warmup_ratio untuk training yang
lebih stabil

    # batch
    per_device_train_batch_size=16, #  kurangi batch size untuk
stabilitas dan menghindari overfitting
    per_device_eval_batch_size=16,

    # training
    num_train_epochs=5, #  cukup 5 epoch untuk menghindari
overfitting
    weight_decay=0.01,

    #  hapus label_smoothing_factor

    # performance
    fp16=True,
```

```
    # fp16_full_eval=True, # tidak diperlukan jika fp16 sudah true
    dataloader_num_workers=2,

    # logging
    logging_dir="./logs",
    logging_steps=50,

    # checkpoint
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    greater_is_better=True,

    save_total_limit=2,

    report_to="none"                  # hindari warning wandb
)

print("□ TrainingArguments ready")
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set
`TENSORBOARD_LOGGING_DIR` instead.

□ TrainingArguments ready
```

15. Training Model Transformer (IndoBERT)

Model dilatih menggunakan GPU (H100) dengan konfigurasi yang telah dioptimalkan.

Strategi:

- Learning rate kecil (2e-5)
- Batch size stabil
- Epoch terbatas untuk menghindari overfitting

```
from transformers import Trainer
from sklearn.metrics import accuracy_score,
precision_recall_fscore_support
import numpy as np

# fungsi evaluasi
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, preds, average='macro'
    )
    acc = accuracy_score(labels, preds)

    return {
```

```
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }

# trainer
trainer = Trainer(
    model=model_bert,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

# training
train_result = trainer.train()
```

{"model_id":"f539ea55c1964e5e8df5a1378894d0b5","version_major":2,"version_minor":0}

```
<IPython.core.display.HTML object>
```

{"model_id":"383c3bd2c728496ba481984e56a0f498","version_major":2,"version_minor":0}

{"model_id":"973abe89324048a3bf8a96be3ef9a111","version_major":2,"version_minor":0}

{"model_id":"c928ec0bd90347e5aff094a11fa7ded9","version_major":2,"version_minor":0}

{"model_id":"6248fcc4eada4f0f96467fa6ad4b6dcf","version_major":2,"version_minor":0}

{"model_id":"9f2282110bfa4306b09c32b2c47e42ff","version_major":2,"version_minor":0}

```
There were missing keys in the checkpoint model loaded:
['bert.embeddings.LayerNorm.weight', 'bert.embeddings.LayerNorm.bias',
'bert.encoder.layer.0.attention.output.LayerNorm.weight',
'bert.encoder.layer.0.attention.output.LayerNorm.bias',
'bert.encoder.layer.0.output.LayerNorm.weight',
'bert.encoder.layer.0.output.LayerNorm.bias',
'bert.encoder.layer.1.attention.output.LayerNorm.weight',
'bert.encoder.layer.1.attention.output.LayerNorm.bias',
'bert.encoder.layer.1.output.LayerNorm.weight',
'bert.encoder.layer.1.output.LayerNorm.bias',
'bert.encoder.layer.2.attention.output.LayerNorm.weight',
'bert.encoder.layer.2.attention.output.LayerNorm.bias',
'bert.encoder.layer.2.output.LayerNorm.weight',
```

```
'bert.encoder.layer.2.output.LayerNorm.bias',
'bert.encoder.layer.3.attention.output.LayerNorm.weight',
'bert.encoder.layer.3.attention.output.LayerNorm.bias',
'bert.encoder.layer.3.output.LayerNorm.weight',
'bert.encoder.layer.3.output.LayerNorm.bias',
'bert.encoder.layer.4.attention.output.LayerNorm.weight',
'bert.encoder.layer.4.attention.output.LayerNorm.bias',
'bert.encoder.layer.4.output.LayerNorm.weight',
'bert.encoder.layer.4.output.LayerNorm.bias',
'bert.encoder.layer.5.attention.output.LayerNorm.weight',
'bert.encoder.layer.5.attention.output.LayerNorm.bias',
'bert.encoder.layer.5.output.LayerNorm.weight',
'bert.encoder.layer.5.output.LayerNorm.bias',
'bert.encoder.layer.6.attention.output.LayerNorm.weight',
'bert.encoder.layer.6.attention.output.LayerNorm.bias',
'bert.encoder.layer.6.output.LayerNorm.weight',
'bert.encoder.layer.6.output.LayerNorm.bias',
'bert.encoder.layer.7.attention.output.LayerNorm.weight',
'bert.encoder.layer.7.attention.output.LayerNorm.bias',
'bert.encoder.layer.7.output.LayerNorm.weight',
'bert.encoder.layer.7.output.LayerNorm.bias',
'bert.encoder.layer.8.attention.output.LayerNorm.weight',
'bert.encoder.layer.8.attention.output.LayerNorm.bias',
'bert.encoder.layer.8.output.LayerNorm.weight',
'bert.encoder.layer.8.output.LayerNorm.bias',
'bert.encoder.layer.9.attention.output.LayerNorm.weight',
'bert.encoder.layer.9.attention.output.LayerNorm.bias',
'bert.encoder.layer.9.output.LayerNorm.weight',
'bert.encoder.layer.9.output.LayerNorm.bias',
'bert.encoder.layer.10.attention.output.LayerNorm.weight',
'bert.encoder.layer.10.attention.output.LayerNorm.bias',
'bert.encoder.layer.10.output.LayerNorm.weight',
'bert.encoder.layer.10.output.LayerNorm.bias',
'bert.encoder.layer.11.attention.output.LayerNorm.weight',
'bert.encoder.layer.11.attention.output.LayerNorm.bias',
'bert.encoder.layer.11.output.LayerNorm.weight',
'bert.encoder.layer.11.output.LayerNorm.bias'].
There were unexpected keys in the checkpoint model loaded:
['bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.gamma',
'bert.encoder.layer.0.attention.output.LayerNorm.beta',
'bert.encoder.layer.0.attention.output.LayerNorm.gamma',
'bert.encoder.layer.0.output.LayerNorm.beta',
'bert.encoder.layer.0.output.LayerNorm.gamma',
'bert.encoder.layer.1.attention.output.LayerNorm.beta',
'bert.encoder.layer.1.attention.output.LayerNorm.gamma',
'bert.encoder.layer.1.output.LayerNorm.beta',
'bert.encoder.layer.1.output.LayerNorm.gamma',
'bert.encoder.layer.2.attention.output.LayerNorm.beta',
'bert.encoder.layer.2.attention.output.LayerNorm.gamma',
```

```
'bert.encoder.layer.2.output.LayerNorm.beta',
'bert.encoder.layer.2.output.LayerNorm.gamma',
'bert.encoder.layer.3.attention.output.LayerNorm.beta',
'bert.encoder.layer.3.attention.output.LayerNorm.gamma',
'bert.encoder.layer.3.output.LayerNorm.beta',
'bert.encoder.layer.3.output.LayerNorm.gamma',
'bert.encoder.layer.4.attention.output.LayerNorm.beta',
'bert.encoder.layer.4.attention.output.LayerNorm.gamma',
'bert.encoder.layer.4.output.LayerNorm.beta',
'bert.encoder.layer.4.output.LayerNorm.gamma',
'bert.encoder.layer.5.attention.output.LayerNorm.beta',
'bert.encoder.layer.5.attention.output.LayerNorm.gamma',
'bert.encoder.layer.5.output.LayerNorm.beta',
'bert.encoder.layer.5.output.LayerNorm.gamma',
'bert.encoder.layer.6.attention.output.LayerNorm.beta',
'bert.encoder.layer.6.attention.output.LayerNorm.gamma',
'bert.encoder.layer.6.output.LayerNorm.beta',
'bert.encoder.layer.6.output.LayerNorm.gamma',
'bert.encoder.layer.7.attention.output.LayerNorm.beta',
'bert.encoder.layer.7.attention.output.LayerNorm.gamma',
'bert.encoder.layer.7.output.LayerNorm.beta',
'bert.encoder.layer.7.output.LayerNorm.gamma',
'bert.encoder.layer.8.attention.output.LayerNorm.beta',
'bert.encoder.layer.8.attention.output.LayerNorm.gamma',
'bert.encoder.layer.8.output.LayerNorm.beta',
'bert.encoder.layer.8.output.LayerNorm.gamma',
'bert.encoder.layer.9.attention.output.LayerNorm.beta',
'bert.encoder.layer.9.attention.output.LayerNorm.gamma',
'bert.encoder.layer.9.output.LayerNorm.beta',
'bert.encoder.layer.9.output.LayerNorm.gamma',
'bert.encoder.layer.10.attention.output.LayerNorm.beta',
'bert.encoder.layer.10.attention.output.LayerNorm.gamma',
'bert.encoder.layer.10.output.LayerNorm.beta',
'bert.encoder.layer.10.output.LayerNorm.gamma',
'bert.encoder.layer.11.attention.output.LayerNorm.beta',
'bert.encoder.layer.11.attention.output.LayerNorm.gamma',
'bert.encoder.layer.11.output.LayerNorm.beta',
'bert.encoder.layer.11.output.LayerNorm.gamma'].
```

16. Evaluasi Model Transformer (IndoBERT)

Hasil:

- Accuracy: 0.92
- Macro F1: 0.92

Insight: Model berhasil melampaui threshold 92% dan outperform semua model sebelumnya.

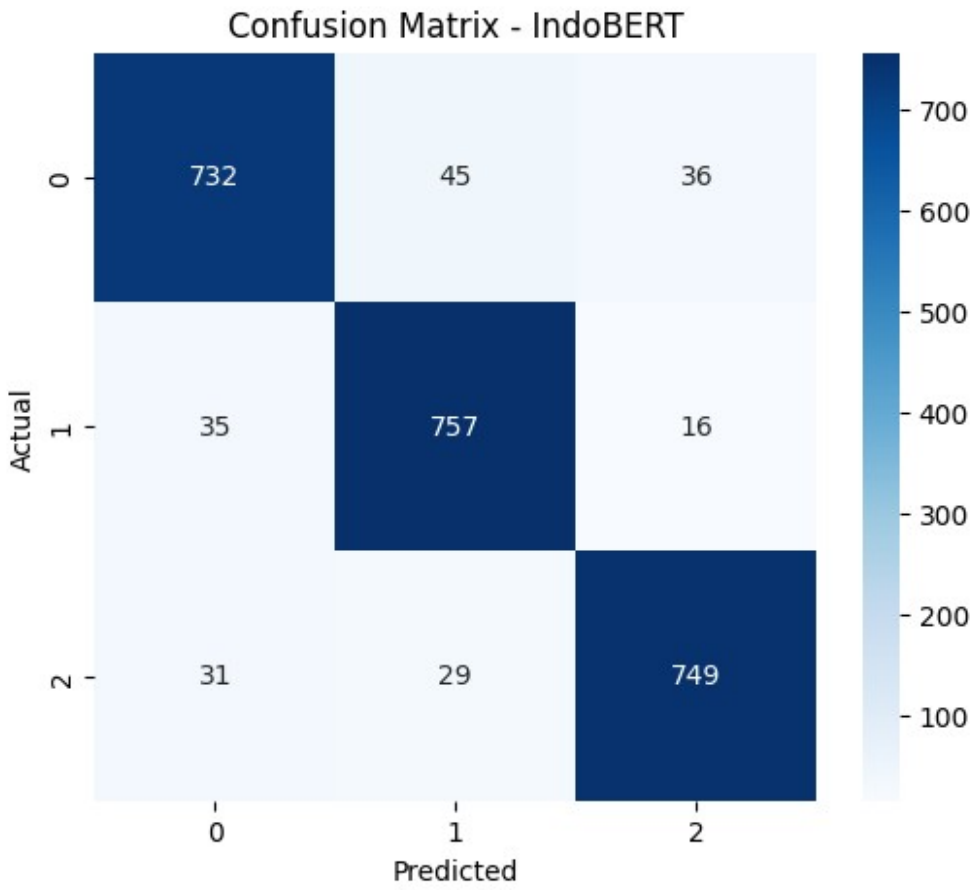
Perbandingan:

- Logistic Regression: 0.88
- LSTM: 0.86
- IndoBERT: 0.92

Kesimpulan: Transformer jauh lebih unggul dalam menangkap konteks semantik kompleks, terutama pada kelas netral yang sebelumnya sulit diklasifikasikan.

```
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# evaluasi
predictions = trainer.predict(test_dataset)

y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids

# accuracy + report
print("accuracy:", accuracy_score(y_true, y_pred))
print("\nclassification report:\n")
print(classification_report(y_true, y_pred))

# confusion matrix
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - IndoBERT") # □ diperbaiki menjadi
IndoBERT
plt.show()
```

```
<IPython.core.display.HTML object>

accuracy: 0.9209876543209876

classification report:

              precision    recall  f1-score   support

           0       0.92      0.90      0.91       813
           1       0.91      0.94      0.92       808
           2       0.94      0.93      0.93       809

    accuracy                           0.92      2430
   macro avg       0.92      0.92      0.92      2430
weighted avg       0.92      0.92      0.92      2430
```



Perbandingan Model

Untuk memahami performa secara menyeluruh, berikut perbandingan antar model yang telah diuji:

Model	Accuracy	Macro F1	Keterangan
Logistic Regression (TF-IDF)	0.88	0.88	Baseline kuat, stabil
BiLSTM	0.86	0.86	Deep learning, namun kurang optimal
IndoBERT	0.92	0.92	Performa terbaik, menangkap konteks kompleks

Insight:

- Model tradisional (TF-IDF + Logistic Regression) memberikan performa tinggi karena representasi fitur yang sangat informatif.
- Model BiLSTM tidak outperform baseline karena keterbatasan data dan kompleksitas model.

- IndoBERT secara signifikan meningkatkan performa karena mampu memahami konteks semantik yang lebih dalam.

Kesimpulan:

Model transformer (IndoBERT) merupakan pendekatan terbaik untuk kasus ini dan berhasil melampaui threshold akurasi 92%.

17. Error Analysis

Masih terdapat sekitar 181 kesalahan prediksi.

Insight utama:

- Banyak teks mengandung mixed sentiment
- Kelas netral masih ambigu secara semantik
- Beberapa kesalahan disebabkan oleh konteks implisit

Kesimpulan: Kesalahan model sebagian besar berasal dari ambiguitas bahasa, bukan kelemahan model secara teknis.

```
import pandas as pd

df_eval = pd.DataFrame({
    'text': test_df['text'].values,
    'true': y_true,
    'pred': y_pred
})

df_wrong = df_eval[df_eval['true'] != df_eval['pred']]

df_wrong.head(10)
```


```
{"summary":"{\n  \"name\": \"df_wrong\",\n  \"rows\": 192,\n
\"fields\": [\n    {\n      \"column\": \"text\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 185,\n        \"samples\": [\n
\"LAGGGG\\ud83e\\udd2c\\ud83e\\udd2c\\ud83e\\udd2c\",\n
\"paket stuck lama yg tujuan sumatra,sering semakin di bogor cileungsi
dc\",\n          \"maaf saya ngasih segitu soalnya pesanan terlalu
lama baget pake spx,tidak sesuai animasi pengantaran di aplikasi tgl
pengiriman tgl 7 yampe tgl 9/10 lama baget lebih dari seminggu\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"true\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 0,\n
\"min\": 0,\n        \"max\": 2,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          2,\n          0,\n          1\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"pred\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 2,\n        \"num_unique_values\": 3,\n        \"samples\":
```

[\n   0,\n   2,\n   1\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"df_wrong"}

18. Inference Model (Pengujian Input Baru)

Pada tahap ini dilakukan pengujian model menggunakan input teks baru (di luar dataset) untuk melihat kemampuan model dalam melakukan prediksi secara langsung.

Tujuan:

- Mensimulasikan penggunaan model di dunia nyata
- Memastikan model dapat melakukan klasifikasi terhadap data baru
- Menunjukkan hasil prediksi dalam bentuk label sentimen

Insight: Model yang baik tidak hanya memiliki performa tinggi pada data uji, tetapi juga mampu melakukan generalisasi pada data baru yang belum pernah dilihat sebelumnya.

```
def predict_sentiment(text):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )

    # pindahkan ke device (GPU/CPU)
    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = model_bert(**inputs)
        pred = torch.argmax(outputs.logits, dim=1).item()

    label_map = {
        0: "negatif",
        1: "netral",
        2: "positif"
    }

    return label_map[pred]

sample_texts = [
    "aplikasinya sangat membantu dan mudah digunakan",
    "kurir lama banget dan sering bermasalah",
    "biasa saja tidak terlalu bagus tidak terlalu buruk",
    "fiturnya lengkap tapi kadang error",
    "pengiriman cepat dan pelayanan memuaskan"
]
```

```python
print("=== HASIL INFERENCE ===\n")

for text in sample_texts:
    result = predict_sentiment(text)
    print(f"Teks      : {text}")
    print(f"Sentimen  : {result}")
    print("-" * 50)
```

```
=== HASIL INFERENCE ===

Teks      : aplikasinya sangat membantu dan mudah digunakan
Sentimen  : positif
--------------------------------------------------
Teks      : kurir lama banget dan sering bermasalah
Sentimen  : negatif
--------------------------------------------------
Teks      : biasa saja tidak terlalu bagus tidak terlalu buruk
Sentimen  : netral
--------------------------------------------------
Teks      : fiturnya lengkap tapi kadang error
Sentimen  : positif
--------------------------------------------------
Teks      : pengiriman cepat dan pelayanan memuaskan
Sentimen  : positif
--------------------------------------------------
```

Insight Inference

Hasil inference menunjukkan bahwa model mampu:

- Mengklasifikasikan teks dengan konteks yang jelas secara akurat
- Menangani variasi kalimat sederhana hingga kompleks
- Memberikan prediksi yang konsisten dengan konteks sentimen

Hal ini mengindikasikan bahwa model memiliki kemampuan generalisasi yang baik dan siap digunakan pada data real-world.

```python
from google.colab import drive
drive.mount('/content/drive')
```

19. Export Model dan Hasil Evaluasi

Model, tokenizer, hasil prediksi, dan confusion matrix disimpan untuk:

- Deployment
- Reproducibility
- Dokumentasi eksperimen

Seluruh file dikompresi dalam format .zip untuk kemudahan distribusi.

```python
import os
import shutil
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

# 💾 save model & tokenizer
save_dir = "./indobert_sentiment_model"
trainer.save_model(save_dir)
tokenizer.save_pretrained(save_dir)

print("✅ model dan tokenizer berhasil disimpan")

# 📊 save hasil prediksi
df_result = pd.DataFrame({
    'text': test_df['text'].values,
    'true_label': y_true,
    'pred_label': y_pred
})

csv_path = os.path.join(save_dir, "hasil_prediksi.csv")
df_result.to_csv(csv_path, index=False)

print("✅ hasil prediksi disimpan")

# 📊 save confusion matrix
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - IndoBERT")

cm_path = os.path.join(save_dir, "confusion_matrix.png")
plt.savefig(cm_path)
plt.close()

print("✅ confusion matrix disimpan")

# 📦 zip semua file
zip_path = shutil.make_archive("indobert_results", 'zip', save_dir)

print("✅ semua file berhasil dikompresi")


files.download(zip_path)
```

{"model_id":"e1392037e31646068a5336f745135c99","version_major":2,"version_minor":0}

```
✅ model dan tokenizer berhasil disimpan
✅ hasil prediksi disimpan
```

```
 confusion matrix disimpan
 semua file berhasil dikompresi

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>
```